

Second-hand Goods Marketplace

1-Week Project Requirement Plan • Next.js · Node.js · TypeScript · MongoDB

■ PROJECT OVERVIEW

Field	Details
Project Name	Second-hand Goods Marketplace (like Facebook Marketplace)
Duration	7 Days (1 Week Sprint)
Frontend	Next.js 14 + TypeScript (App Router)
Backend	Node.js + Express + TypeScript
Database	MongoDB + Mongoose ODM
Auth Solution	NextAuth.js v5 (best choice — see Auth section below)
File Storage	Cloudinary (for product images)
Styling	Tailwind CSS

■ WHY NextAuth.js v5 (Best Auth Choice)

Reason	Explanation
Built for Next.js	Works natively with App Router, Server Components & API routes
Multiple providers	Email/password, Google, GitHub — just config, no extra code
Session management	Handles JWT & database sessions automatically
MongoDB adapter	Official adapter syncs users directly into your MongoDB
Beginner-friendly	No manual JWT signing, token refresh or cookie logic needed
vs Clerk	Clerk is easier but paid beyond limits; NextAuth is free & teaches real auth
vs Custom JWT	Custom JWT teaches more but takes 3-4 days alone — too slow for 1 week

■ TECH STACK AT A GLANCE

Layer	Technology	Purpose
-------	------------	---------

Frontend	Next.js 14 + TypeScript	Pages, routing, server components, SSR
Styling	Tailwind CSS	Utility-first responsive design
Auth	NextAuth.js v5	Sign up / login / session handling
Backend	Node.js + Express	REST API for listings, users, messages
Database	MongoDB + Mongoose	Store users, listings, messages, chats
Images	Cloudinary	Upload & serve product photos
Validation	Zod	Type-safe form & API validation
HTTP client	Axios / fetch	Frontend → Backend API calls

■ MONGODB SCHEMA DESIGN

Collection	Key Fields
users	_id, name, email, password (hashed), avatar, location, createdAt
listings	_id, title, description, price, condition, category, images[], seller (ref:users), location, status (active/sold), createdAt
messages	_id, conversationId, sender (ref:users), text, createdAt
conversations	_id, participants[] (ref:users), listing (ref:listings), lastMessage, updatedAt

■ 7-DAY BUILD PLAN

DAY

1

Setup &
Auth

■ FRONTEND (Next.js + TypeScript)

- Init Next.js 14 project with TypeScript & Tailwind CSS
- Set up folder structure: app/, components/, lib/, types/
- Install NextAuth.js v5 and configure MongoDB adapter
- Build Sign Up page (name, email, password) with Zod validation
- Build Login page with error handling
- Create Navbar component with auth state (login/logout button)
- Protect routes using NextAuth middleware (middleware.ts)

■ *Focus: get auth working end-to-end before anything else.*

■ BACKEND (Node.js + MongoDB)

- Init Node.js + Express project with TypeScript (tsconfig)
- Connect MongoDB with Mongoose — test connection
- Create User schema/model (name, email, password, avatar)
- POST /api/auth/register — hash password with bcrypt, save user
- Test all routes with Postman or Thunder Client
- Set up .env for MONGODB_URI, NEXTAUTH_SECRET, CLOUDINARY keys

DAY

2

Listings —
Create &
View

■ FRONTEND (Next.js + TypeScript)

- Build Create Listing form (title, price, condition, category, description)
- Integrate Cloudinary upload widget for product images (multi-image)
- Show image preview before submission
- Build Listings homepage — grid of listing cards
- Build Listing Detail page (/listings/[id]) with all info + seller contact button
- Add loading skeletons for cards

■ *Milestone: you can post an item and see it live on the homepage.*

■ BACKEND (Node.js + MongoDB)

- Create Listing schema/model (all fields + seller ref)
- POST /api/listings — create new listing (auth required)
- GET /api/listings — fetch all active listings (paginated, 12 per page)
- GET /api/listings/:id — single listing with seller info (populate)
- Cloudinary image upload endpoint — return secure URLs
- Add input validation with Zod on all routes

DAY

3

Search,
Filter & Categories

■ FRONTEND (Next.js + TypeScript)

- Build Search bar in Navbar — debounced input (300ms)
- Build Filter sidebar: category, price range (min/max), condition
- Apply filters as URL query params (?category=electronics&maxPrice;=5000)
- Read query params in Next.js page and pass to API
- Show 'No results found' empty state with illustration
- Highlight search term in listing titles (optional bonus)

■ Key concept: MongoDB query building with dynamic filters.

■ BACKEND (Node.js + MongoDB)

- Update GET /api/listings to accept: search, category, condition, minPrice, maxPrice
- Use MongoDB regex for text search on title & description
- Use MongoDB range queries for price filtering
- Add index on 'category' and 'status' fields for performance
- Return total count alongside results for pagination
- Test all filter combinations in Postman

DAY

4

User
Profile &
My
Listings

■ FRONTEND (Next.js + TypeScript)

- Build User Profile page (/profile/[id]) — avatar, name, location, listings
- Build My Listings dashboard — list user's own listings with edit/delete
- Build Edit Listing page — pre-fill form with existing data
- Add Mark as Sold button — changes listing status
- Show sold badge on listing cards
- Add avatar upload on profile settings page

■ Key concept: ownership checks — only the seller can edit/delete.

■ BACKEND (Node.js + MongoDB)

- GET /api/users/:id — public profile with their active listings
- GET /api/users/me/listings — current user's all listings (auth required)
- PUT /api/listings/:id — update listing (only owner can edit)
- DELETE /api/listings/:id — delete listing (only owner)
- PATCH /api/listings/:id/sold — mark as sold
- PUT /api/users/me — update profile, avatar URL

DAY

5

Messaging System

■ FRONTEND (Next.js + TypeScript)

- Add Contact Seller button on listing detail page
- Build Conversations list page (/messages) — all chats
- Build Chat detail page (/messages/[conversationId])
- Show messages with sender name + timestamp
- Auto-scroll to latest message
- Show listing thumbnail in conversation header

■ Key concept: designing relational data (users ↔ listings ↔ conversations).

■ BACKEND (Node.js + MongoDB)

- Create Message & Conversation schemas
- POST /api/conversations — start a new conversation about a listing
- GET /api/conversations — get all conversations for current user
- GET /api/conversations/:id/messages — get messages in a conversation
- POST /api/conversations/:id/messages — send a message
- Prevent duplicate conversations for same listing + buyer pair

DAY

6

Wishlist,
Polish &
UX

■ FRONTEND (Next.js + TypeScript)

- Add Save / Wishlist heart button on listing cards
- Build Saved Listings page — /wishlist
- Add toast notifications (react-hot-toast) for all actions
- Make the app fully responsive (mobile-first check)
- Add 404 page and error boundaries
- Add listing image carousel on detail page
- Performance: lazy load images with next/image

■ Focus: clean UX details that make the project feel real & complete.

■ BACKEND (Node.js + MongoDB)

- Add savedBy[] array field to Listing schema
- POST /api/listings/:id/save — toggle save/unsave
- GET /api/users/me/saved — get user's saved listings
- Add proper HTTP error responses (400, 401, 403, 404)
- Add rate limiting on POST routes (express-rate-limit)
- Review all API responses — consistent JSON format

DAY

7

Testing, Deployment
&
README

■ FRONTEND (Next.js + TypeScript)

- Test all pages and flows end-to-end manually
- Fix any broken links, missing loading states, or edge cases
- Deploy frontend to Vercel (free) — add all .env vars in dashboard
- Test deployed app on mobile browser
- Write a clean README with screenshots + setup instructions
- Record a short demo video (optional but great for portfolio)

■ *Goal: a live, working URL you can share with anyone.*

■ BACKEND (Node.js + MongoDB)

- Deploy backend to Railway or Render (free tiers)
- Make sure CORS is set to allow your Vercel domain
- Check all environment variables are set in deployment dashboard
- Test deployed API with Postman against production URL
- Add basic error logging (console.error or a simple logger)
- Final review: no hardcoded URLs, no exposed secrets

■ COMPLETE API ENDPOINTS REFERENCE

Method + Endpoint	Auth?	Description
POST /api/auth/register	No	Register new user
POST /api/auth/login	No	Login (handled by NextAuth)
GET /api/listings	No	Get all listings (search + filter)
POST /api/listings	Yes	Create new listing
GET /api/listings/:id	No	Get single listing + seller info
PUT /api/listings/:id	Yes	Update listing (owner only)
DELETE /api/listings/:id	Yes	Delete listing (owner only)
PATCH /api/listings/:id/sold	Yes	Mark listing as sold
POST /api/listings/:id/save	Yes	Save / unsave listing
GET /api/users/:id	No	Get public user profile
PUT /api/users/me	Yes	Update own profile
GET /api/users/me/listings	Yes	Get my listings
GET /api/users/me/saved	Yes	Get my saved listings
GET /api/conversations	Yes	Get my conversations
POST /api/conversations	Yes	Start new conversation
GET /api/conversations/:id/messages	Yes	Get messages in conversation
POST /api/conversations/:id/messages	Yes	Send a message

■ KEY CONCEPTS YOU WILL LEARN

Day	Concept	Why It Matters
1	Authentication with NextAuth.js	Every real app needs auth — this is the #1 skill
2	Image uploads with Cloudinary	File storage is a core backend skill
2	REST API design	Standard for all web apps
3	MongoDB dynamic queries + indexing	Filtering & search is on every job requirement

4	Role-based access / ownership checks	Security — only owners can edit their data
5	Relational data in MongoDB	Real apps have connected data across collections
6	Error handling + UX polish	What separates a demo from a real product
7	Deployment (Vercel + Railway)	You need a live URL for your portfolio

■ RECOMMENDED DAILY TIME BUDGET

Activity	Time
Backend (Node.js API + MongoDB)	2 hours
Frontend (Next.js + TypeScript)	2.5 hours
Testing & debugging	30 min
Reading docs / learning	30 min
Total per day	~5.5 hours

■ RECOMMENDED npm PACKAGES

Package	Side	Use
next-auth	Frontend	Authentication
@auth/mongodb-adapter	Frontend	Sync NextAuth users to MongoDB
zod	Both	Schema validation
axios	Frontend	HTTP requests to backend
react-hot-toast	Frontend	Toast notifications
cloudinary	Backend	Image upload handling
mongoose	Backend	MongoDB ODM
bcryptjs	Backend	Password hashing
express	Backend	HTTP server framework
cors	Backend	Allow cross-origin requests
dotenv	Backend	Environment variables
express-rate-limit	Backend	Prevent API abuse

■ By end of Week 1 you will have: A live marketplace where users can sign up, post items with photos, search & filter listings, message sellers, and save favourites — deployed on Vercel + Railway with a real MongoDB database.